

比特币：点对点电子现金系统

中本聪
satoshin@gmx.com
bitcoin.org

摘要。 纯粹的点对点版本的电子现金将允许在线款项可直接从一方汇给另一方，无需通过金融机构。数字签名是解决方案的一部分，但最主要的是如果仍然需要可信的第三方来防止双重支出，则利益就会丧失。我们使用点对点网络提出了双重支出问题的解决方案。网络通过将交易哈希成一个持续的链来给交易加上时间戳基于哈希的工作量证明，形成不重做就无法更改的记录工作量证明。最长的链不仅可以作为序列的证明目睹了事件，但证明它来自最大的 CPU 功率池。如只要大部分 CPU 功率由不合作的节点控制攻击网络，他们将生成最长的链条并超过攻击者。这个网络本身需要最小的结构。尽最大努力广播消息基础，节点可以随意离开和重新加入网络，接受最长的时间工作量证明链作为他们离开时发生的事情的证据。

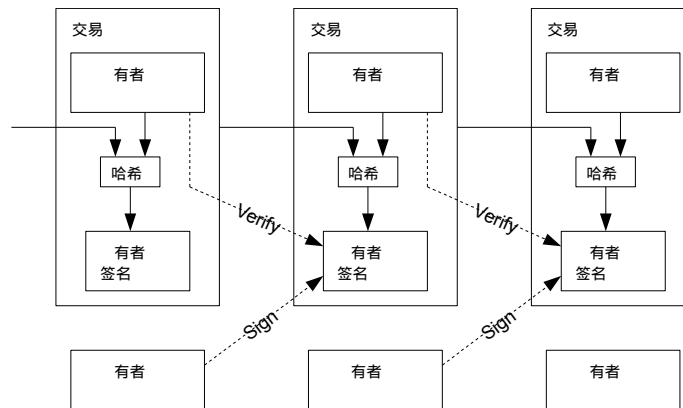
1. 引言

互联网上的商业几乎完全依赖于金融机构充当可信第三方处理电子支付。虽然该系统运行良好大多数交易，它仍然存在基于信任的模型固有的弱点。完全不可逆的交易实际上是不可能的，因为金融机构不能避免调解争议。调解成本增加了交易成本，限制了最低实际交易规模，切断小额临时交易的可能性，失去不可逆转地支付不可逆转付款的能力会带来更大的代价，可逆服务。由于存在逆转的可能性，对信任的需求随之蔓延。商家必须警惕他们的客户，为他们提供比他们原本需要的更多的信息。一定比例的欺诈行为被认为是不可避免的。这些成本和付款不确定性可以通过使用实物货币亲自避免，但不存在付款机制通过没有可信方的通信渠道。

需要的是基于密码证明而不是信任的电子支付系统，允许任何两个有意愿的当事方直接进行交易，而无需信任第三方。在计算上不切实际的交易将保护卖方免受欺诈，并且可以轻松实施常规的托管机制来保护买家。在本文中，我们提出了一种使用点对点分布式方法来解决双重支出问题的解决方案时间戳服务器，用于生成交易时间顺序的计算证明。这个只要诚实的节点共同控制的CPU功率比其他任何节点都多，系统就是安全的攻击者节点的合作小组。

。 交易

我们将电子 币定 为数字签名链。 个 有者将 币转
接 来，对 一 交易的哈希 和 一个 有者的 进行数字签名
并将它们 加 币的 。 款 可以 证签名以 证链
有 。



当然，问题在于 款 无法 实其中一 有者没有重 支出
币。一个常 的解决方案是 入一个可信的中 机构 币 来 一个机构
双重消 的交易。 交易，必须将 币 给 币
发行新 币，只有直接从 币 发行的 币 可以被重 使用。
这种解决方案的问题在于， 个货币 系的 运 决于
币， 交易都必须通过 币，就 行一 。

我们需要一种方法 款 道以 的 有者没有签字
交易。就我们的目的而言，最 的交易 是最重要的交易， 以我们不在乎
于 来 双重支出。确认没有交易的 一方法是

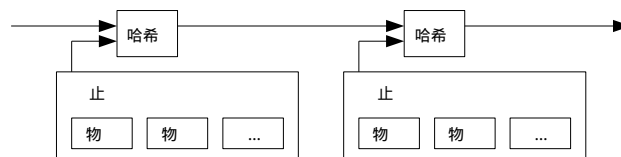
意 有交易。在基于 币 的模型中， 币 道 有交易，
决定 个 。要在没有可信方的情 实现这一目 ，交易必须
开 布，我们需要一个 者就 一 成共 的系统

它们的顺序。 款 需要证明在 交易时，
大多数节点都认为这是第一个 的。

时间戳服务器

我们提出的解决方案从时间戳服务器开 。时间戳服务器的工作原理是
对要加时间戳的项目 进行哈希处理，并广 发布该哈希，例如在

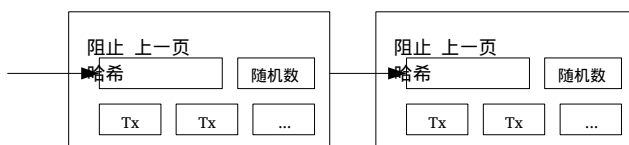
Us n t 子 。时间戳证明数据一定存在于
然，是时 进入哈希了。 个时间戳都 之 的时间戳
它的哈希 形成一条链， 增加一个时间戳都会增 之 的时间戳。



4. 工作量证明

要在点对点的基础上实现分布式时间戳服务器，我们将需要使用证明-下班系统类似于亚当·巴克的 Hashcash [6]，而不是报纸或 Usenet 的帖子。工作量证明包括扫描一个值，该值在进行哈希处理时，例如使用 SHA-256 时，哈希以零位数开头。所需的平均工作量是指数级的需要零位，可以通过执行单个哈希来验证。

对于我们的时间戳网络，我们通过增加随机数来实现工作量证明阻塞直到找到一个为区块的哈希值提供所需的零位的值。一旦 CPU 已花费精力使其满足工作量证明，区块无法更改无需重做工作。由于后来的方块被链接在后面，因此需要更改方块将包括重做之后的所有方块。



工作量证明还解决了在多数决策中确定代表性的问题制作。如果多数是建立在“一IP地址一票”的基础上的，那么任何人都可能颠覆它能够分配许多 IP。工作量证明本质上是一个 CPU 一票。大多数决策由最长的链条代表，该链所投入的工作量证明工作量最大在里面。如果大部分 CPU 功率由诚实节点控制，则诚实的链将增长速度最快，比任何竞争连锁店都要快。要修改过去的区块，攻击者必须重做方块及其后所有方块的工作量证明然后赶上并超越诚实节点的工作。稍后我们将展示速度较慢的攻击者追上来的可能性随着后续区块的添加，呈指数级递减。

为了补偿硬件速度的提高以及随着时间的推移对运行节点的兴趣的变化，工作量证明难度由移动平均线确定，目标是平均数为每小时区块数。如果生成速度太快，难度就会增加。

5. 网络

运行网络的步骤如下：

- 1) 新交易将广播到所有节点。
- 2) 每个节点将新交易收集到一个区块中。
- 3) 每个节点都在努力为其区块寻找困难的工作量证明。
- 4) 当一个节点找到工作量证明时，它会将该区块广播到所有节点。
- 5) 只有当区块中的所有交易均有效且尚未花费时，节点才会接受该区块。
- 6) 节点通过努力在区块中创建下一个区块来表达对该区块的接受链，使用接受区块的哈希值作为先前的哈希。

节点始终将最长的链视为正确的链，并将继续工作扩展它。如果两个节点同时广播下一个区块的不同版本，则有些节点可以先接收一个或另一个。在这种情况下，他们正在研究他们收到的第一个，但是保存另一个分支以防它变得更长。下次证明时平局就会被打破-找到工作中断，一个分支变长；在另一个分支上运行的节点然后分支将切换到更长的分支。

新的交易广播不一定需要到达所有节点。只要他们到达多节点，不之后它们就会进入一个方块。区块广播可以线。如果一个节点有收到区块，它将在收到下一个区块时该区块，到它过了一个。

。

例，区块中的第一交易是一交易，它会动有的新硬由区块的创建者创建。这增加了节点支网络的动力，并提供了一种最将硬分到通中的方法，因为有行硬的中机。

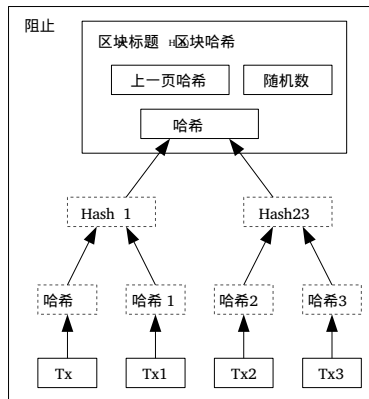
步增加一定数量的新硬类似于开者的支增加通的。就我们而，的是CPU时间力。

可以通过交易费来。如果交易的值为小于其入值，是交易费，该费用加在值中包交易的区块。一旦入了定数量的硬通，可以过到交易费，是通费。

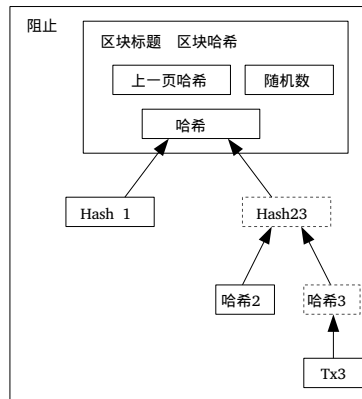
这种可能有于节点保诚实。如果的攻击者能够的CPU功率比所有诚实的节点都要多，他必须在使用它之间做通过他的或用它来生成新硬来他人。他该现则更有可，这的则有于他使用更多的新硬了破制度他的有效性，其他所有人都而为一。

。 收 间

一旦硬中的最新交易被在足够多的区块之下，之前花费的交易就是可以将其以节间。为了在不破区块哈希值的情况下进这一点，交易在克[] [2] [5]中进行哈希处理，区块的哈希中包。然后，可以通过来实方块。部哈希值确实如此不需要存。



在 e e 中哈希处理的交易

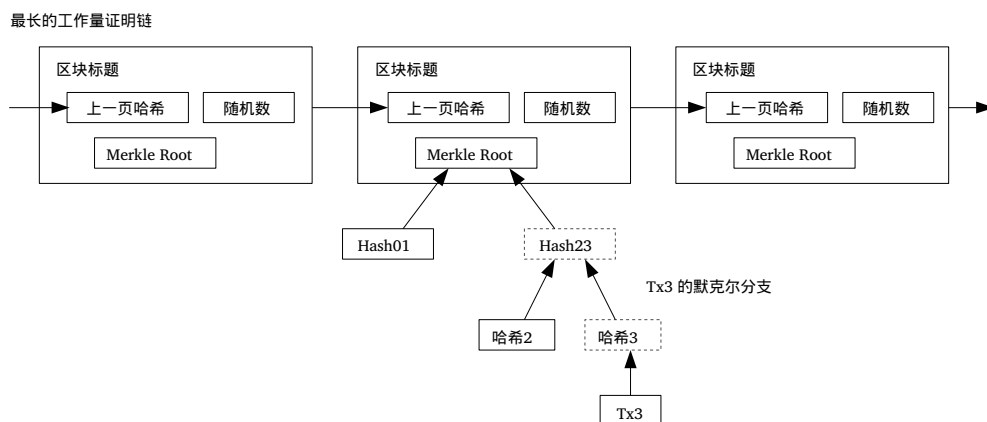


方块中 Tx -2 之后

有交易的区块标头为节。如果我们方块是每1分生成一次，节624365每4.2。使用机系统2，通以2的存，定当前的增长率为每1.2，使必须保区块标头，存间不会成为问题。

8. 简化的付款验证

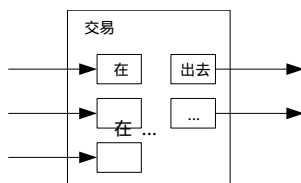
无需运行完整的网络节点即可验证付款。用户只需要保留最长的工作量证明链的区块标头的副本，他可以通过查询获得网络节点直到他确信自己拥有最长的链，然后获得 Merkle 分支将交易与带有时间戳的区块相关联。他无法查看这笔交易他本人，但是通过将其链接到链中的某个位置，他可以看到网络节点已经接受了它，并在它进一步确认网络已接受之后添加屏蔽。



因此，只要诚实的节点控制网络，验证是可靠的，但更可靠如果网络被攻击者压倒，则易受攻击。虽然网络节点可以验证交易本身，这种简化的方法可能会被攻击者捏造的欺骗只要攻击者能够继续压制网络，交易就会持续下去。一种策略是防范这种情况的方法是在网络节点检测到无效时接受来自网络节点的警报阻止，提示用户的软件将完整区块和提醒的交易下载到确认不一致。经常收到付款的企业可能仍然希望运行自己的节点以获得更独立的安全性和更快的验证。

9. 合并和拆分值

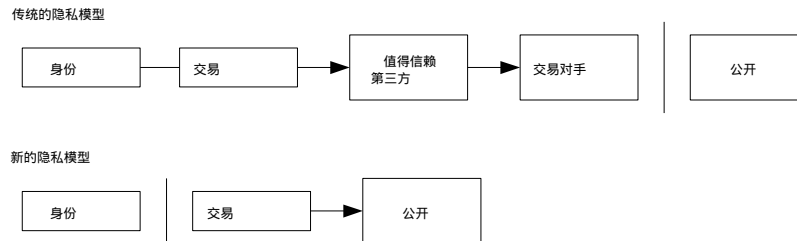
尽管可以单独处理硬币，但制作硬币会很笨拙转账中每分钱进行单独交易。为了允许对值进行拆分和组合，交易包含多个输入和输出。通常，要么只有一个输入来自较大的先前交易或合并较小金额的多个输入，最多两笔输出：一个用于付款，一个将零钱（如果有）退还给发件人。



应该注意的是，扇出，其中一笔交易依赖于几笔交易，而这些交易取决于更多，这里不是问题。从来没有必要提取交易历史记录完整独立副本。

10. 隐私

传统的银行模式通过限制对信息的访问来实现一定程度的隐私。所涉各方和可信第三方。必须公开宣布所有交易。排除了这种方法，但仍然可以通过中断信息流来维护隐私。另一个地方：通过保持公钥匿名。公众可以看到有人在发送给他人的金额，但没有将交易与任何人联系起来的信息。这是类似于证券交易所发布的信息水平，其中的时间和规模个别交易，即 录像带，是公开的，但没有透露当事方是谁。



作为额外的防火墙，应为每笔交易使用新的密钥对以保留它们从关联到共同所有者。使用多输入时，某些链接仍然是不可避免的交易，这必然表明他们的投入归同一个所有者所有。风险是，如果密钥的所有者被泄露，关联可能会泄露属于该密钥的其他交易同一个所有者。

11. 计算

我们考虑的场景是攻击者试图以比实际更快的速度生成备用链。即使这样做了，它也不会使系统遭受任意更改，例如比如凭空创造价值或拿走从来不属于攻击者的钱。节点是不会接受无效的交易作为付款，诚实的节点也永远不会接受区块包含它们。攻击者只能尝试更改自己的一笔交易来收回他最近花的钱。

诚实链和攻击链之间的竞赛可以说是二项式随机漫步。成功事件是将诚实的链条扩展了一个区块，从而增加了其区块以 1 领先，失败事件是攻击者的链条被扩展了一个区块，从而减少了差距 1。

攻击者从给定赤字中追赶的概率与赌徒的概率类似毁灭问题。假设一个拥有无限积分的赌徒一开始就有赤字，可能玩的是进行无限次的试验，试图达到盈亏平衡。我们可以计算出他曾经的概率达到盈亏平衡，或者攻击者曾经追上了诚实的链条，如下所示：

p 诚实的节点找到下一个区块的概率
 q 攻击者找到下一个方块的概率
 q_z 攻击者从后面的 z 个方块追上来的概率

$$q_z = \begin{cases} 1 & \text{如果 } p < q \\ \left(\frac{q}{p}\right)^z & \text{如果 } p > q \end{cases}$$

鉴于我们的假设 $p < q$ ，概率随方块数的增加而呈指数级下降
 攻击者必须赶上增加。如果他不走运，就算对他不利
 尽量向前猛冲，随着他进一步落后，他的机会变得越来越小。

我们现在考虑新交易的接收者需要等多久才能成为
 足够确定发件人无法更改交易。我们假设发件人是攻击者
 谁想让收款人相信他向他付款了一段时间，然后将其改为还款给
 一段时间后他自己已经过去了。发生这种情况时将提醒接收者，但是
 发件人希望为时已晚。

接收方生成新的密钥对，并在不久前将公钥提供给发送者
 签署。这可以防止发件人通过以下方式提前准备区块链
 一直持续到他有幸提前走得足够远，然后在以下位置执行交易
 那一刻。交易发送后，不诚实的发件人开始秘密地进行交易
 包含他交易的替代版本的平行链。

收款人会等到交易被添加到区块中，然后 z 区块已经被封锁了
 在它之后链接。他不知道攻击者取得的确切进展，但是
 假设诚实的区块占用了每个区块的平均预期时间，则攻击者的潜力
 进度将是具有预期值的泊松分布：

$$= \frac{z \cdot q}{p}$$

为了确定攻击者现在还能赶上的概率，我们将泊松密度乘以
 他本可以取得的每一个进步都取决于他能从那时起赶上的可能性：

$$\sum_{i=0}^{\infty} \frac{e^{-\lambda} \lambda^i}{i!} \cdot \begin{cases} \frac{q}{p} & \text{如果 } i \leq z \\ 1 & \text{如果 } i > z \end{cases}$$

重新排列以避免对分布的无限尾求和...

$$1 - \sum_{i=0}^{z-1} \frac{e^{-\lambda} \lambda^i}{i!} \left(1 - \frac{q}{p} \right)^{z-i}$$

正在转换为 C 代码...

```
#include
double attackerSuccessProbability ( 双 q , 整数 z )
{ 双精度 p = 1.0-q;
double lambda = z * (q/p);
双和 = 1.0 ;
int i, k;
对于 (k = 0; k <= z; k++)
{
双泊松 = exp (-lambda);
对于 (i = 1; i <= k; i++)
泊松 *= lambda/i;
总和 += 泊松 * (1-pow (q/p, z-k));
}
返回总和 ;
}
```

运行一些结果，我们可以看到 z 的概率呈指数下降。

```
q=0.1
z=0 P=1.0000000
z=1 P=0.2045873
z=2 P=0.0509779
z=3 P=0.0131722
z=4 P=0.0034552
z=5 P=0.0009137
z=6 P=0.0002428
z=7 P=0.0000647
z=8 P=0.0000173
z=9 P=0.0000046
z=10 P=0.0000012
```

```
q=0.3
z=0 P=1.0000000
z=5 P=0.1773523
z=10 P=0.0416605
z=15 P=0.0101008
z=20 P=0.0024804
z=25 P=0.0006132
z=30 P=0.0001522
z=35 P=0.0000379
z=40 P=0.0000095
z=45 P=0.0000024
z=50 P=0.0000006
```

求解小于 0.1% 的 P...

```
P < 0.001
q=0.10 z=5
q=0.15 z=8
q=0.20 z=11
q=0.25 z=15
q=0.30 z=24
q=0.35 z=41
q=0.40 z=89
q=0.45 z=340
```

12. 结论

我们提出了一个不依赖信任的电子交易系统。我们一开始就是由数字签名制成的硬币的常用框架，可以很强烈地控制所有权，但如果无法防止双重支出，则不完整。为了解决这个问题，我们提议使用工作量证明来记录公开交易历史的点对点网络。如果是真实的节点，攻击者在计算上很快就会变得不切实际。控制大部分 CPU 功率。该网络的非结构化简单性非常强大。节点只需很少的协调即可同时完成所有工作。它们不需要被识别，因为消息是不配送到任何特定地点，只需要尽最大努力交付即可。节点可以随意离开并重新加入网络，接受工作量证明链作为证明发生在他们不在的时候。他们用自己的 CPU 能力投票，表示接受通过努力扩展区块来有效区块，通过拒绝处理来拒绝无效区块。任何必要的规则和激励措施都可以通过这种共识机制来执行。

1. , 1998 。
2. 和 . . . q , 以最小的成 计 的时
信任要求, 在 20 信息论 会 , 1999 5 。
3. , 如何为数字 加时 , 于 , 3 , 有
2, 99 111 , 1991 。
4. , 提 数字时 的效率和可 性 ,
在 通信 和计算机 法 , 329 334 , 1993 。
5. , 的 名 , 在 ACM 会议论
论计算机和通信 , 28 35 , 1997 4 。
6. 一种拒绝 对 ,
. , 2002。
7. .C. , 公 系统协议 , 于 Proc. 1980 会
, 计算机 会 , 122 133 , 1980 4 。
8. , 概率论 用概论 , 1957 。